System Design: Sliding-Window Page Classification

1. Problem and Approach

Classifying each page of a large PDF as **Invoice, Purchase Order, Contract, Appendix, or Other** can be difficult when pages are processed one by one. For example, a continuation page may contain only line items or contract clauses, without the original title.

To solve this, we use a sliding-window approach. For every target page, the model receives:

- Previous page text and its predicted label.
- Current page text, which is the only page to classify.
- Next page text for extra context.

The system prompt tells the model to classify only the current page. The previous label is just a hint, not a fixed rule, because a new document may start on any page.

2. Processing Flow

The classifier.py file handles the main process, including payload creation, API calls, validation, retries and logging.

For each document, pages are processed in order:

1. Build the payload using the previous, current and next page.
2. Call the LLM and check its JSON response.
3. If the label is valid, save it and use it as context for the next page.
4. If the call fails after retries, save the page as failed and use UNKNOWN as the previous label for the next page.
5. Continue processing instead of stopping the whole document.

The source page text is always read from the original input. Only the validated previous label is carried forward.

The response must contain exactly one key, label, with a value from the allowed categories. Invalid responses and temporary API errors are retried up to the configured retry limit.

Page boundaries are also handled: the first page has no previous page, and the last page has no next page.

For multiple documents, we can run different documents in parallel. However, pages inside one document must remain sequential because each prediction depends on the previous page's result. A concurrency limit, retry backoff and idempotent database writes can help handle provider limits and avoid duplicate results.

3. Latency and Token Cost

The main drawback of this design is latency. A 100-page PDF needs around 100 sequential LLM calls.

The estimated processing time is:

Total time ≈ Number of pages × Average time per call

Pages	1.2 sec/call	1.8 sec/call
16	19 sec	29 sec
100	120 sec	180 sec

In the demo, a synthetic 16-page document was processed using gpt-4o-mini through OpenRouter. With MAX_RETRIES=1, all 16 API calls succeeded, and the predicted labels matched the expected labels. Total time was around 30 seconds.

So, a 100-page document may take around 3 minutes, without counting extra retry delays.

The sliding window also uses more tokens than single-page classification. If each page contains around T tokens and the prompt adds S tokens, the input can be roughly **3T + S**

per page for pages with both neighbors available. The target page keeps its full text, while LimitPageText limits long page content.

Some ways to improve performance are:

- Run multiple documents in parallel.
- Use a smaller model after checking its accuracy.
- Use HTTP keep-alive and suitable timeouts.
- Shorten neighbor text while keeping important headers and context.
- Track token usage and use prompt caching when supported.

4. Reliability and Testing

The project includes retries for temporary errors and invalid model responses. Unit tests use mocked API calls to check label parsing, page boundaries, failed pages and whether processing continues after a failure.

The demo uses demo_data.py as its expected output. It includes examples of invoices, purchase orders, contracts, appendices and other pages. The main.py file reports matched labels, unmatched pages and failures.

For evaluation, we should also test with real PDFs and a larger labeled dataset. Macro-F1 and accuracy on continuation pages will help us understand if the approach is working properly.

5. Deployment and Next Steps

The current implementation includes the orchestrator, prompt and payload design, demo, tests, logging and evaluation support.

A full production deployment is not completed yet. PDF text extraction, durable job storage and the Cloudflare Worker setup(**Used OpenRouter instead of Cloudfare Worker**) are still pending.

A possible deployment is to use **Cloudflare Workers AI** for classification and D1 for storing page results and checkpoints. For larger deployments, PostgreSQL and a durable queue can provide better reliability.

My recommendation is to keep pages sequential within each document and scale by processing different documents in parallel. The next priority should be testing on more

real-world PDFs, followed by adding persistent checkpoints and the Cloudflare deployment.